Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3 — Benchmark Report

NYC Yellow Taxi Data | Two-worker clusters | October 2026

Abstract. Apache Spark and Ray Data were compared on a real distributed preprocessing workload built from monthly NYC Yellow Taxi Parquet files. The measured directory contained 32 files, 119,595,677 input rows and 1,940.21 MB of Parquet data; the taxi-zone lookup CSV was used for enrichment. Because the monthly files changed from 19 to 20 and then 21 columns, the benchmark used the 19 fields common to every file. The common pipeline was timestamp cleaning, null filtering, duration and distance checks, exact duplicate removal, taxi-zone enrichment, average-speed calculation and Snappy Parquet export. Spark completed the selected benchmark in 300.539 s and Ray in 562.993 s, with 115,570,482 output rows in each case. Spark was approximately 1.87× faster on this two-worker setup.

1 Setup and Dataset

The experiment used two Docker-based clusters with two workers each. Spark used a standalone master at `spark://spark-master:7077`; Ray used a head node at `ray-head:6379`. The cluster, shared data mounts, worker services and network communication were configured manually. Spark's Master UI confirmed two active workers, while the Ray Dashboard showed the head and two workers alive.

Table 1: Cluster configuration.

Framework	Version	Workers	Cores/worker	Memory/worker	Total cores
Spark	3.5.6	2	4	12 GiB	8
Ray	2.59.0	2	4	12 GiB	8

The local dataset inventory actually used for the benchmark spans January 2024 through August 2026. It contains 32 monthly files and 1,940.21 MB of Parquet data, giving a working dataset of roughly 2 GB including the lookup table. Later monthly files contain additional fields, so the benchmark deliberately retained only the 19-column intersection. This avoids one framework receiving extra fields simply because a later month has a newer schema.

2 Common Pipeline and Implementation

The two programs follow the same business sequence:

`ingest` → `clean` → `exact deduplicate` → `location join` → `speed feature` → `Parquet export`

Cleaning converts the pickup and dropoff timestamps, removes rows missing required fields, computes trip duration, rejects non-positive distance or duration, and derives duration in minutes and pickup hour. Exact duplicates are defined using the complete 19-column benchmark record. Spark applies `dropDuplicates(TRIP_COLUMNS)`; Ray hashes those fields into buckets only to route rows and then performs exact `drop_duplicates` within each bucket. Thus the hash is a routing mechanism, not the definition of equality.

The taxi-zone table is a small lookup keyed by `LocationID`. Spark broadcasts it to avoid a large fact-table shuffle. Ray performs a distributed key join and exposes the number of join partitions as a parameter. The derived speed is

$$\text{avg_speed_mph} = \frac{\text{trip distance}}{\text{duration in seconds}/3600}.$$

After deduplication, Ray narrows the large fact table to 11 trip fields before the join. This was done to reduce the amount of data carried through the expensive shuffle. Spark keeps the 19 retained source fields. This is the main implementation difference that affects final schema width; both selected outputs nevertheless contain the same number of rows.

3 Benchmark Results

The benchmark stopwatch covers ingestion through completion of the Parquet write. Output validation is performed afterwards. The selected successful runs were:

Table 2: Selected end-to-end results.

Metric	Spark	Ray
Input files	32	32
Input rows	119,595,677	119,595,677
Output rows	115,570,482	115,570,482
Ingestion → export	**300.539 s**	**562.993 s**
Output Parquet files	128	209

Spark is 262.454 s faster. The Ray/Spark time ratio is 562.993/300.539 ≈ 1.87, so Spark uses about 46.6% less wall-clock time in the selected runs. The matching output row count shows that the same number of records survived

the cleaning and duplicate-removal logic; because the final Ray projection is narrower, it is not equivalent to a claim of identical output schemas.

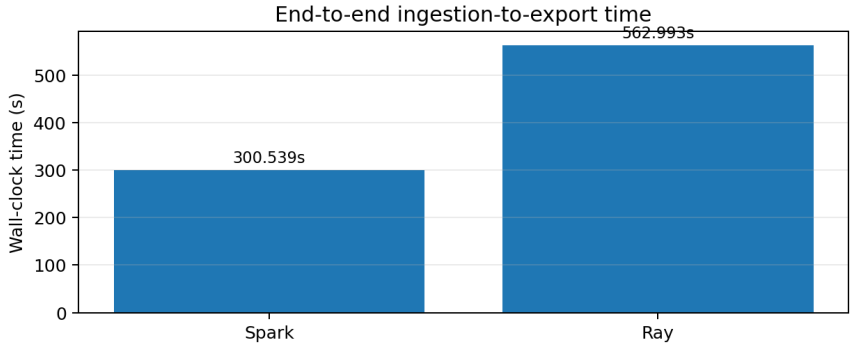


Figure 1: Selected end-to-end ingestion-to-export time. Lower is better.

4 Resource Behaviour and Ray Tuning

Container-level monitoring was collected while the distributed jobs were running. Docker reports CPU as aggregate usage across the cores assigned to a container, so values above 100% indicate concurrent use of multiple cores.

Table 3: Observed resource peaks from the retained monitoring traces.

Node	Peak CPU	Peak observed memory
Spark worker 1	965.86%	1.899 GiB
Spark worker 2	1040.49%	1.925 GiB
Ray worker 1	661.12%	∼3.9 GiB
Ray worker 2	649.61%	∼6.6 GiB

The traces are workload-monitoring samples rather than framework-internal counters, and the Ray samples are not tightly time-aligned to the 562.993 s selected run. A stronger run-correlated observation from Ray is its memory-pressure event: a node reached about 13.83 GB of a 14.43 GB limit. Ray then killed workers and retried shuffle-reduce tasks. The task path at that point was the join reduce, followed by the speed transformation and write.

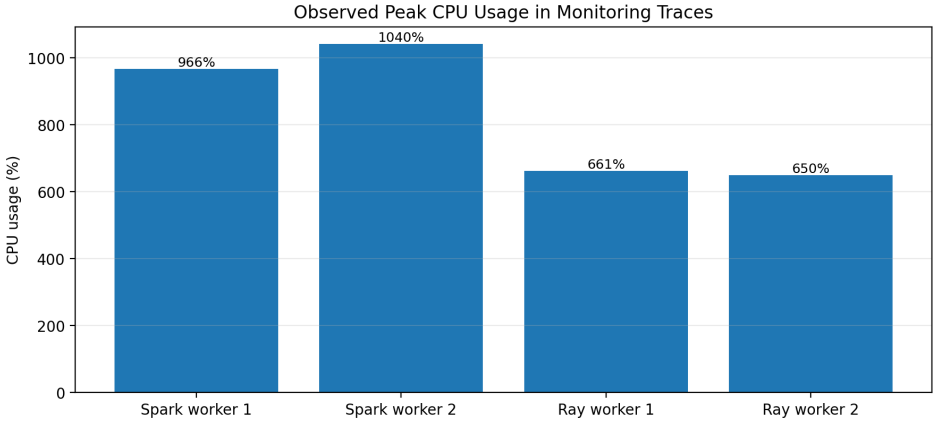


Figure 2: Observed peak CPU values in the container-monitoring traces.

Ray's initial read stage was deliberately bounded to eight output blocks and four concurrent readers. Shuffle V2, disk-backed hash shuffle, isolated read workers, and projection before the join were used after the earlier runs showed memory pressure. Join partitioning was then varied.

Table 4: Ray join-partition experiment.

Configuration	Time (s)	Output rows	Observation
32 partitions	536.110	115,570,482	OOM/retries observed
256 partitions	547.221	115,570,482	OOM/retries remained
Later 256-partition run	562.993	115,570,482	selected final run

The directly comparable 32-to-256 change added 11.111 s, about 2.1%, and did not remove the memory-pressure pattern. Ray's documentation describes key-based joins as shuffle operations over distributed blocks and describes

disk-based shuffle as useful when shuffled data is large relative to object-store memory [3,4]. This matches the behaviour seen in the logs: the expensive operation was the join shuffle/reduce rather than file reading alone.

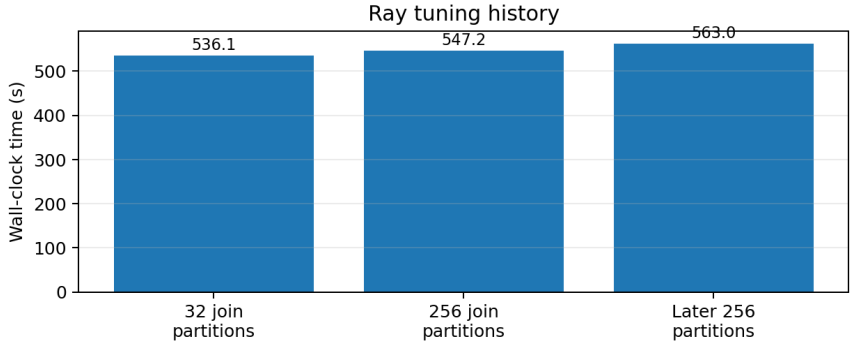


Figure 3: Ray tuning runs.

5 Python UDF Measurement

A separate 1,000,000-row probe measured the cost of the custom speed calculation. Spark's native arithmetic was compared with the scalar Python UDF:

Table 5: Spark UDF probe on 1,000,000 real rows.

Measurement	Time (s)
Native Spark arithmetic	0.234
Python UDF	1.184
Reported overhead	406.90%

The Python UDF took approximately 5.06 times as long as the native expression. Spark documents Python UDFs as a separate Python execution path from the JVM-side SQL expression engine [2], so the measured difference is consistent with the additional Python execution/serialization path. This number is a result of this cluster and probe, not a universal Spark constant.

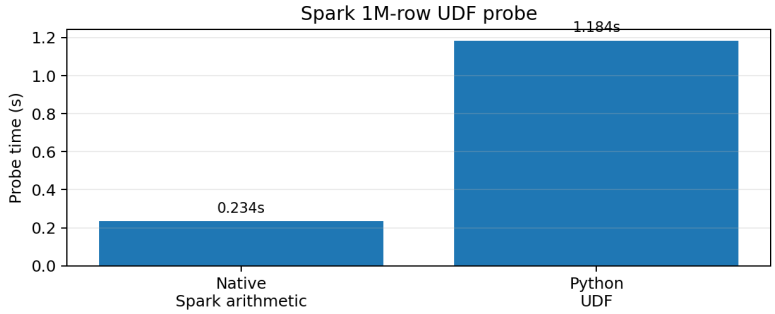


Figure 4: Native Spark arithmetic versus the scalar Python UDF.

The Ray UDF-probe job also completed, but the final numeric native-versus-Python timing output was not retained. It is therefore not reported as a percentage here. The Dashboard duration for that job is not a UDF-only timing because the current program constructs and executes the main pipeline before calling the probe.

6 Interpretation and Conclusion

The result is consistent with the roles of the two systems. Spark's SQL/DataFrame engine is aimed at structured data processing and can optimize operations such as projection, filtering and joins as a query plan [2]. Ray Data is designed for scalable data processing in AI workflows and provides Python batch transformations directly [3]. The present workload is dominated by structured ETL: duplicate removal, a large keyed join, shuffle and Parquet output. It contains no model training or inference stage. Spark performing better on this workload is therefore reasonable, but the experiment does not justify a general claim that Spark is always faster than Ray.

The most significant difference observed during execution was memory behaviour. The Ray join redistributed the large trip table across the two workers. During the reduce phase, several tasks were active together and node memory approached the configured limit. The 13.83/14.43 GB event was followed by worker kills and retries. Increasing join partitions from 32 to 256 changed the size and number of partitions but did not remove the pressure. This was the main reason the Ray run took longer in this environment. Ray's documentation describes key-based joins as shuffle operations and notes that disk-based shuffle is intended for large shuffles where object-store pressure becomes important [3,4].

The Spark resource trace shows sustained multi-core use on both workers, with observed peaks of 965.86% and 1040.49%. The corresponding memory observations were about 1.9 GiB per worker. The Ray monitoring trace showed higher observed memory on its workers, while the direct Ray logs also recorded the near-limit node condition. These numbers are container-level measurements and are used to describe the behaviour of this experiment rather than to claim a universal hardware-efficiency ranking.

6.1 Python UDF Interpretation

The UDF probe illustrates a different layer of the comparison. The Spark expression that computes speed can execute inside Spark's normal SQL expression path, whereas the scalar Python UDF invokes Python code. On 1,000,000 real rows, the native expression took 0.234 s and the Python UDF took 1.184 s, giving a reported overhead of 406.90%. The result shows why a Python UDF can become a visible cost in an otherwise structured Spark pipeline. Ray also executes Python functions as data transformations, but the retained experiment did not preserve the numerical Ray UDF timings, so no numerical cross-framework UDF claim is made.

6.2 AI Use and Manual Work

The implementation used a limited LLM-assisted workflow. Most of the Spark pipeline was written first, and the Ray implementation was then translated from the same logic. LLM assistance was used for syntax translation, reading runtime errors and suggesting targeted changes. Examples that were actually tested include limiting Ray reads to 8 blocks with 4 concurrent readers, enabling Shuffle V2 and disk-backed shuffle, projecting the fact table before the join, and changing join partitions from 32 to 256. The Docker network, shared volumes, worker services, ports, Spark master connection, and Ray head/worker connections were configured manually. The programs were run repeatedly, errors were inspected in the terminal and UI, and the reported numbers were taken from successful executions rather than generated by the LLM.

6.3 Limitations and Final Result

The selected runs produced identical output row counts of 115,570,482, but the current Ray final projection is narrower than Spark's final projection. Exact final-schema parity is therefore not established. The selected Ray result used 256 join partitions, so that option should be specified when reproducing the 562.993 s result. CPU and memory values are container-level observations from monitoring traces.

For this workload, Spark is the practical winner:

$$\text{Spark} = 300.539\,s, \qquad \text{Ray} = 562.993\,s.$$

Spark is approximately 1.87× faster and uses about 46.6% less wall-clock time. For a BI-first structured ETL workload, the measured result favours Spark. For an AI-first pipeline combining distributed data processing with Python-heavy inference or ML training, Ray remains a reasonable alternative because its execution model is designed around those workloads. The experiment itself, however, supports the stronger claim only for the workload and cluster used here.

Performance Tuning Note. The code was tuned after observing actual execution behaviour. Ray's read concurrency and block count were reduced to control memory pressure; disk-backed Shuffle V2 was enabled; the fact table was narrowed before the join; and join partition counts were tested at 32 and 256. Spark used a broadcast lookup join, adaptive execution, and bounded Parquet file sizing. All performance figures in this report come from the executed benchmark runs and UI/terminal measurements.

References. [1] DA3408, "A3: Spark vs. Ray - The Data Engineering Duel," course assignment handout, 2026. [2] Apache Spark Documentation, "Spark SQL, DataFrames and Datasets" and Python UDF documentation, Spark 3.5.x. [3] Ray Documentation, "Ray Data: Scalable Data Processing for AI Workloads," Ray 2.59.x. [4] Ray Documentation, "Ray Data Internals" and join/shuffle documentation, Ray 2.59.x.

Appendix — UI and Execution Evidence

The first two figures are the screenshots explicitly required for the final submission. The remaining figures document the completed Spark application, the full Ray run, the UDF probe, and the final Ray output verification.

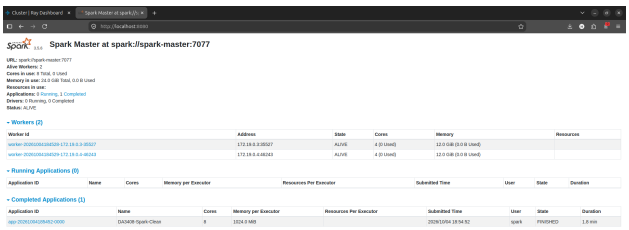


Figure 5: Spark Master UI: two active workers.

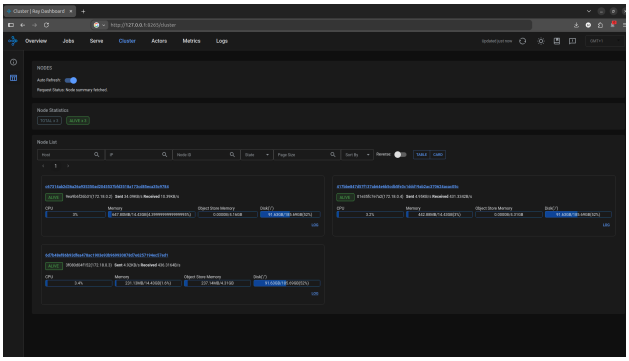


Figure 6: Ray Dashboard Cluster: head plus two workers.

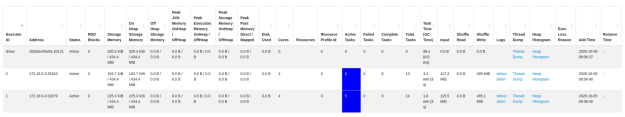


Figure 7: Spark 4040 Executors view.

	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(3)	0	709.3 KiB / 1.3 GiB	709.3 KiB / 1.3 GiB	0.0 B / 0.0 B	0.0 B	8	10	0	17	27	2.6 min (7 s)	232.8 MiB	0.0 B	980 MiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(3)	0	709.3 KiB / 1.3 GiB	709.3 KiB / 1.3 GiB	0.0 B / 0.0 B	0.0 B	8	10	0	17	27	2.6 min (7 s)	232.8 MiB	0.0 B	980 MiB	0

Figure 8: Spark 4040 executor summary.

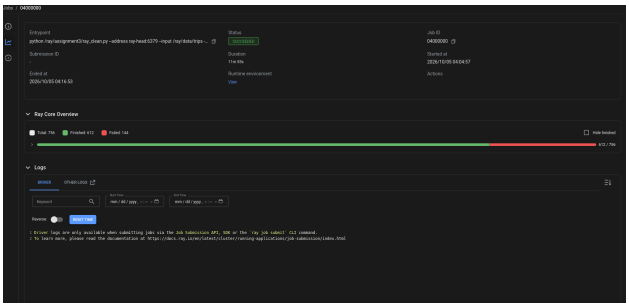


Figure 9: Ray Dashboard during the full 119.6M-row dataset run.

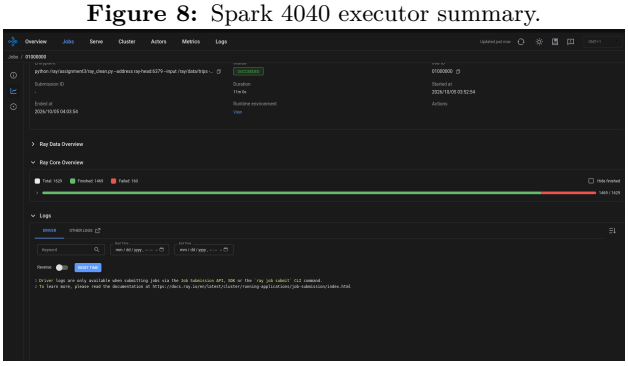


Figure 10: Ray Dashboard during the 1M-row UDF probe run.

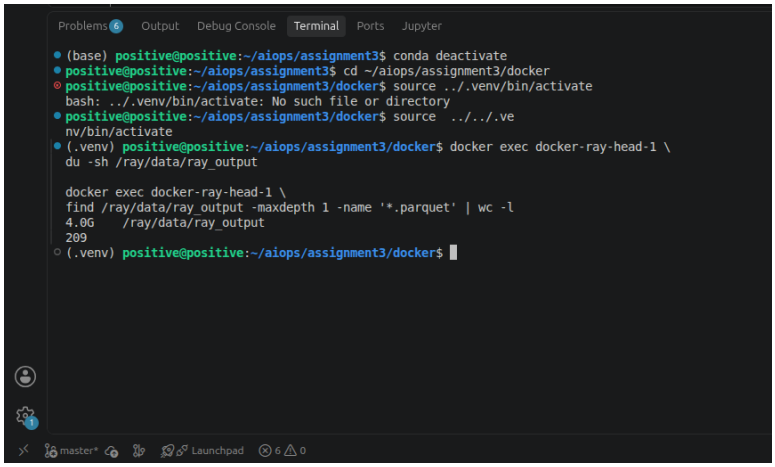


Figure 11: Terminal verification of the Ray output: 4.0 GB and 209 Parquet files.